

Sistema Multi-Agente para la Gestión y Optimización de Carteras de Inversión en Renta Fija y Variable

José Carlos Carbone¹, MARCO SEBASTIAN PADILLA¹, Nilayvi Stephani Barrientos Pérez¹, and Federico Hernan Sanches¹

¹Maestría en Inteligencia Artificial

Marzo 2026

Resumen

Este trabajo presenta el diseño e implementación de un asistente financiero inteligente basado en una arquitectura de micro-agentes coordinados. El sistema integra datos en tiempo real de activos de renta variable (Acciones) y renta fija (Obligaciones Negociables - ON), permitiendo al usuario final interactuar mediante lenguaje natural a través de la plataforma Telegram. La solución no solo centraliza la visualización del portfolio, sino que aplica modelos de optimización de varianza mínima, análisis de sentimiento de noticias y simulaciones estocásticas para proyectar flujos de fondos y proponer rebalances estratégicos.

1. Introducción

El acceso a mercados financieros ha crecido exponencialmente, sin embargo, la complejidad técnica para analizar instrumentos de deuda privada (ONs) y acciones locales e internacionales sigue siendo una barrera para el inversor promedio. Este proyecto propone un asistente que utiliza Modelos de Lenguaje de Gran Escala (LLM) no solo como interfaz, sino como orquestador de herramientas cuantitativas.

El objetivo principal es resolver la asimetría de información mediante tres pilares:

- **Retrospectiva:** Análisis de costo de oportunidad frente a benchmarks.
- **Proyección:** Estimación de flujos futuros y volatilidad.
- **Optimización:** Sugerencia de carteras eficientes basadas en el perfil de riesgo.

2. Arquitectura del Sistema

Se ha optado por una arquitectura de *Sistema Multi-Agente* (MAS) implementada mediante el framework *LangGraph*. Esta decisión permite desacoplar la lógica de negocio de la obtención de datos y el procesamiento de lenguaje natural.

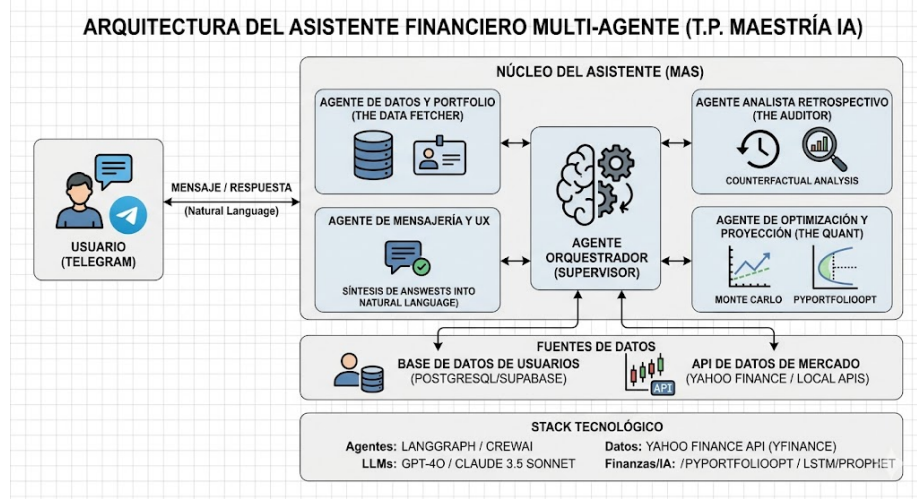


Figura 1: Arquitectura del Sistema Multi-Agente propuesto.

2.1. Agente Orquestador (The Supervisor)

Actúa como el nodo central de decisión. Utiliza *Function Calling* para clasificar la intención del usuario y dirigir el flujo hacia los agentes especialistas.

2.2. Agentes Especialistas

1. **Data Fetcher:** Encargado de la persistencia en PostgreSQL y la ingesta de datos vía APIs (yfinance) y Web Scraping para ONs.
2. **The Auditor:** Calcula el rendimiento histórico comparado con el Dólar MEP y el índice S&P 500.
3. **The Quant:** El motor matemático. Utiliza la librería *PyPortfolioOpt* para la frontera eficiente y modelos de Monte Carlo para proyecciones.
4. **The News Scout:** Implementa técnicas de RAG y análisis de sentimiento sobre noticias financieras de última hora.

3. Metodología de Optimización

Para la optimización de la cartera, el sistema se basa en la teoría moderna de selección de cartera. Dado un conjunto de activos con retornos esperados $E[R]$ y una matriz de covarianza Σ , se busca minimizar la varianza de la cartera:

$$\text{mín } w^T \Sigma w$$

Sujeto a:

$$\sum w_i = 1, \quad w_i \geq 0$$

En el caso de las Obligaciones Negociables, el sistema prioriza el cálculo de la Tasa Interna de Retorno (TIR) y la *Duration* para mitigar el riesgo de tasa de interés.

4. Sinergia Multi-Agente: News Scout y Optimización Quant

La innovación de este sistema radica en la retroalimentación entre el análisis cualitativo y el cuantitativo. Mientras que el *Agente Quant* opera sobre series de tiempo históricas para calcular la matriz de covarianza Σ , el *News Scout* proporciona un "ajuste de sentimiento" (α_{sent}) que modifica los retornos esperados en tiempo real.

El modelo de retorno ajustado se define como:

$$E[R_{adj}] = E[R_{hist}] \cdot (1 + \lambda \cdot s)$$

Donde $s \in [-1, 1]$ es el score de sentimiento derivado del procesamiento de lenguaje natural (NLP) de noticias financieras y λ es un factor de sensibilidad al mercado. Esta integración permite que el sistema detecte eventos de cisne negro.º noticias corporativas críticas (ej. cambios en la calificación crediticia de una ON) antes de que se reflejen plenamente en el precio histórico.

5. Desarrollo Técnico

El stack tecnológico seleccionado comprende Python 3.11+, utilizando *Aio-gram* para la interfaz de Telegram y *OpenAI/Anthropic* como modelos de razonamiento. La integración de los agentes se realiza mediante grafos dirigidos acíclicos (DAG), garantizando que la respuesta final sea sintetizada por un agente de UX que traduce métricas complejas a un lenguaje comprensible.

6. Resultados esperados/Conclusión

La implementación de un sistema multi-agente permite una escalabilidad superior a los chatbots lineales tradicionales. La capacidad de integrar análisis

de noticias con modelos econométricos tradicionales posiciona a este asistente como una herramienta integral para la toma de decisiones financieras en tiempo real.